LINUX SIGNAL-BASED PROCESS CONTROLLER

A functional Ubuntu process-control prototype using C and POSIX signals

OSSP HACKATHON | PROJECT PRESENTATION

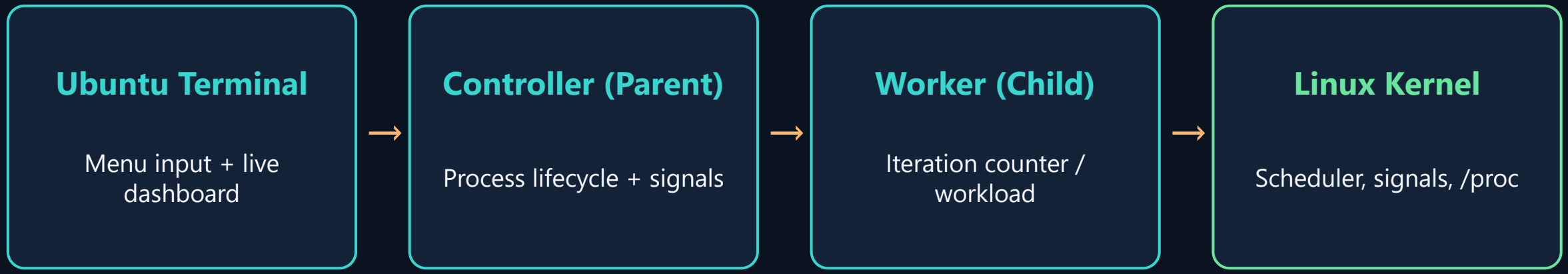
Team members:
NAEENI SATWIKA - 2520090151
SAMEEKSHA - 2520090229
NANDINI - 2520090230

Problem Statement

How can users control a running Linux process and observe its actual state?

- Develop a process-control utility for Ubuntu Linux.
- Support pause, resume, terminate, and restart operations using Linux signals.
- Make process-state changes observable, not just confirmation messages.
- Provide a simple, educational demonstration of OS process management.

System Architecture



Parent and child are separate Linux processes; a shared memory region exposes worker progress to the dashboard.

Implementation Approach

- Create a child worker with fork().
- Use kill(pid, SIGSTOP) to suspend the worker.
- Use kill(pid, SIGCONT) to resume it from the same execution point.
- Use SIGTERM for graceful shutdown; waitpid() reaps the child.
- Restart by terminating the old worker and creating a new PID.
- Refresh the terminal dashboard and inspect /proc/PID/status.

Technology Stack

OPERATING SYSTEM

Ubuntu Linux / WSL environment

PROGRAMMING

C language with GCC compiler

OS INTERFACES

POSIX signals, fork(), waitpid(), mmap()

MONITORING

/proc filesystem and ANSI terminal refresh

Testing Strategy

Start	New worker PID; counter increases
Pause	SIGSTOP; counter freezes; Linux state T
Resume	SIGCONT; same PID; counter continues
Terminate	Worker exits; no active PID
Restart	New PID; counter resets
Invalid input	Error message; application stays usable

Results and Evidence

OBSERVED IN INITIAL PROTOTYPE

Worker created with PID 1137; pause/resume commands acknowledged; termination recorded; restarted worker used PID 1248; logs showed a new iteration sequence.

UPGRADED DASHBOARD

Replacement C code adds shared counter and /proc-based status, CPU and memory readings. These upgraded features should be verified in Ubuntu before claiming final test success.

Recommended demo evidence: screenshots of Running → Paused (T) → Resumed, plus changed PID after Restart.

Impact and Applications

- Makes Linux process management visible and interactive for OS learners.
- Shows practical effects of kernel-managed signals on real processes.
- Helps demonstrate parent–child coordination and process lifecycle.
- Useful as a teaching prototype for Operating Systems labs and hackathons.
- Can be extended with multi-process support, alerts, and process history.